academia global

Universidad Corporativa Coppel | Programas Académicos

Ingeniería en Desarrollo de Software

Actividad | #2 | Programa Banco Mexicano (parte 1) Lenguaje de Programación IV

Ingeniería en Desarrollo de Software

TUTOR: Aarón Iván Salazar Macías.

ALUMNO: Alberth Alexis Rodriguez Astorga

FECHA: 01 de Abril del 2026

Indice

Introducción……...03

Descripción……....03

Justificación……...03

Investigación……..04

Desarrollo……..06

Conclusión……...13

Referencias……...…….13

Introducción

Hoy en día, los sistemas bancarios forman parte importante de la vida cotidiana, ya que permiten a las personas realizar operaciones como depósitos, retiros y consultas de saldo de manera rápida y segura.

Detrás de estas acciones existe software que organiza la información y garantiza que cada operación se realice correctamente.

El desarrollo de programas que simulan este tipo de servicios ayuda a comprender cómo funcionan los sistemas reales, especialmente en lo relacionado con el manejo de dinero y la interacción con el usuario.

Además, permite aplicar conceptos de la programación orientada a objetos, que facilitan la organización del código y hacen posible crear sistemas más claros, ordenados y reutilizables.

En este contexto, crear un programa que represente operaciones básicas de un banco no solo permite practicar la lógica de programación, sino también entender la importancia de diseñar soluciones que sean eficientes, seguras y fáciles de usar para las personas.

Descripción

La actividad consiste en desarrollar un programa en Java que simule el funcionamiento básico de un sistema bancario, permitiendo a los usuarios realizar operaciones sencillas como depósitos, retiros y consulta de saldo, el programa debe iniciar con un menú principal en el que el usuario pueda seleccionar alguna de estas opciones mediante el teclado, facilitando la interacción de manera clara y ordenada.

En el caso de la opción de depósito, el sistema deberá solicitar la cantidad a ingresar y sumarla al saldo disponible, además de preguntar al usuario si desea realizar otro depósito, repitiendo el proceso en caso afirmativo o regresando al menú principal en caso contrario, para la opción de retiro, el programa deberá pedir la cantidad a retirar y descontarla del saldo, regresando posteriormente al menú principal.

Es importante considerar que el desarrollo debe realizarse utilizando el paradigma de la programación orientada a objetos, organizando el programa en clases, atributos y métodos que representen adecuadamente las operaciones de una cuenta bancaria.

Asimismo, se deben emplear estructuras de control como condicionales y ciclos para manejar las decisiones del usuario y la repetición de procesos.

Cabe mencionar que en esta etapa de la actividad se trabaja únicamente con las opciones de depósito y retiro, dejando la implementación de la consulta de saldo y la salida del programa para una fase posterior.

De esta manera, la actividad permite desarrollar habilidades fundamentales en programación, al mismo tiempo que simula un problema real del entorno bancario.

Justificación

La realización de esta actividad utilizando Java y un entorno de desarrollo integrado se justifica porque estas herramientas permiten desarrollar programas de manera estructurada, clara y eficiente.

Java, al ser un lenguaje orientado a objetos, facilita la organización del código mediante clases y métodos, lo que es ideal para simular sistemas como el bancario.

Además, el uso de un entorno de programación ayuda a detectar errores, probar el funcionamiento del programa y mejorar la productividad durante el desarrollo.

En conjunto, estas herramientas permiten aplicar de forma práctica los conceptos teóricos aprendidos, logrando una solución funcional y bien organizada para el problema planteado.

Investigación

Investiga un ejemplo de cómo declarar una clase con sus propiedades y métodos en cada uno de los siguientes lenguajes de programación:

1.Java:

```
class Persona {
   String nombre;
   int edad;

   void saludar() {
      System.out.println("Hola, mi nombre es " + nombre);
   }
}
```

2.Python:

```
class Persona:
   def __init__(self, nombre, edad):
      self.nombre = nombre
      self.edad = edad

   def saludar(self):
      print("Hola, mi nombre es", self.nombre)
```

3.C#:

```
class Persona {
   public string Nombre { get; set; }
   public int Edad { get; set; }

   public void Saludar() {
      Console.WriteLine("Hola, mi nombre es " + Nombre);
   }
}
```

4.C++

```
#include <iostream>
using namespace std;

class Persona {
public:
   string nombre;
   int edad;
```

```
    void saludar() {
        cout << "Hola, mi nombre es " << nombre << endl;
    }
};
```

5.PHP

```
#include <iostream>
using namespace std;

class Persona {
public:
    string nombre;
    int edad;

    void saludar() {
        cout << "Hola, mi nombre es " << nombre << endl;
    }
};
```

Lenguaje	Palabra Clave	Inicialización / Atributos	Referencia Interna	Ejemplo de Método
Java	class	Se definen fuera de los métodos.	this	public void saludar() { ... }
Python	class	Dentro del método __init__.	self	def saludar(self):
C#	class	Usa propiedades con get; set;.	this	public void Saludar() { ... }
C++	class	Requiere secciones public: o private:.	this->	void saludar() { ... }
PHP	class	Las variables usan el símbolo $.	$this->	public function saludar() { ... }

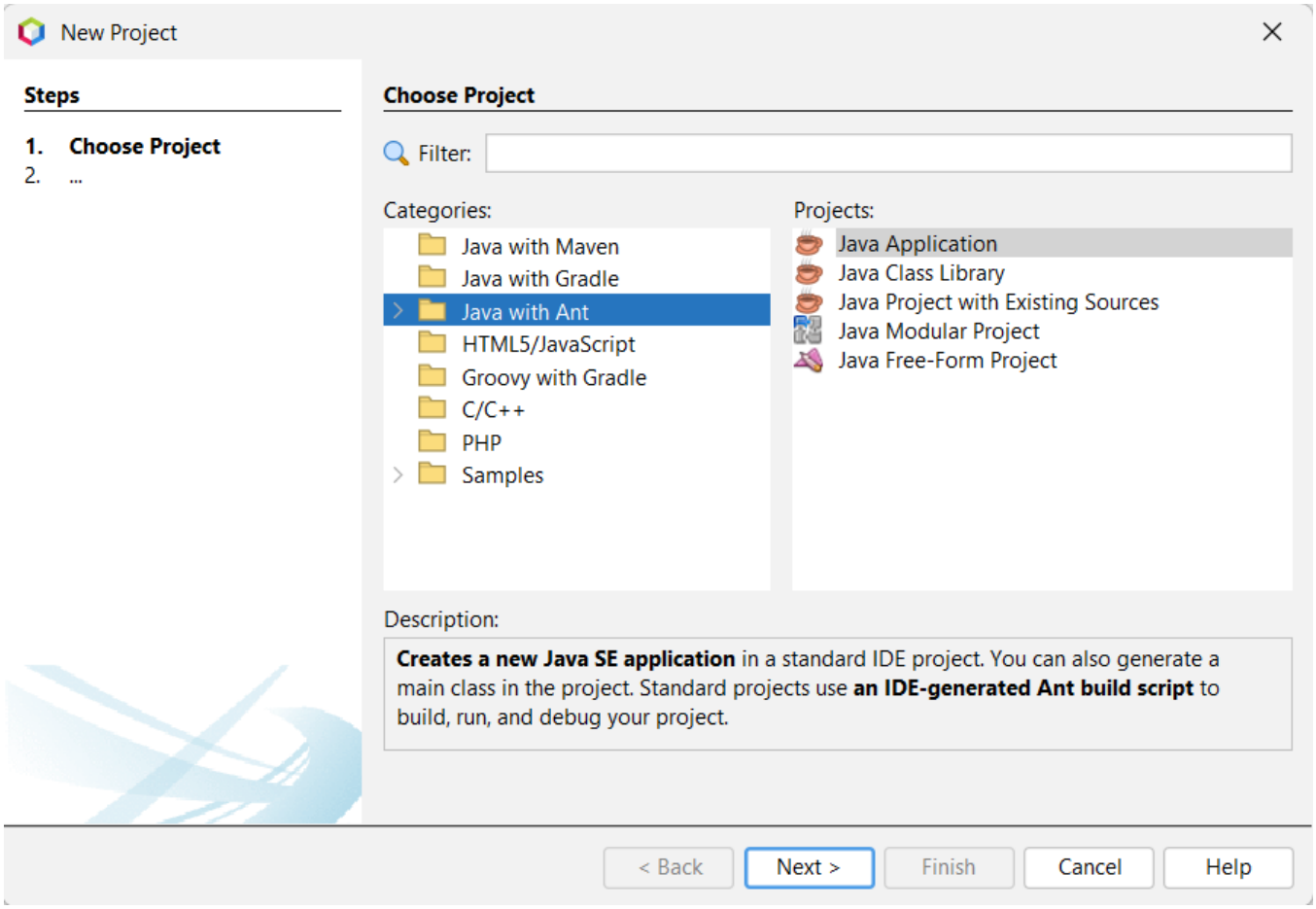
Aunque la sintaxis varíe entre lenguajes, el concepto de clase permanece como el molde fundamental de la POO, presentando diferencias marcadas: mientras que Java y C# guardan una gran similitud al usar modificadores de acceso (public, private) para cada elemento y llaves para estructurar el código, Python se distancia por completo al eliminar las llaves en favor de la sangría (indentación) y requerir obligatoriamente el parámetro self en sus métodos.

Por otro lado, C++ destaca por ser el único que exige un punto y coma ; inmediatamente después de cerrar la llave de la clase, y PHP se diferencia por el uso de la flecha -> y el símbolo $ para interactuar con las propiedades, contrastando con el operador punto . predominante en la mayoría de los demás lenguajes.

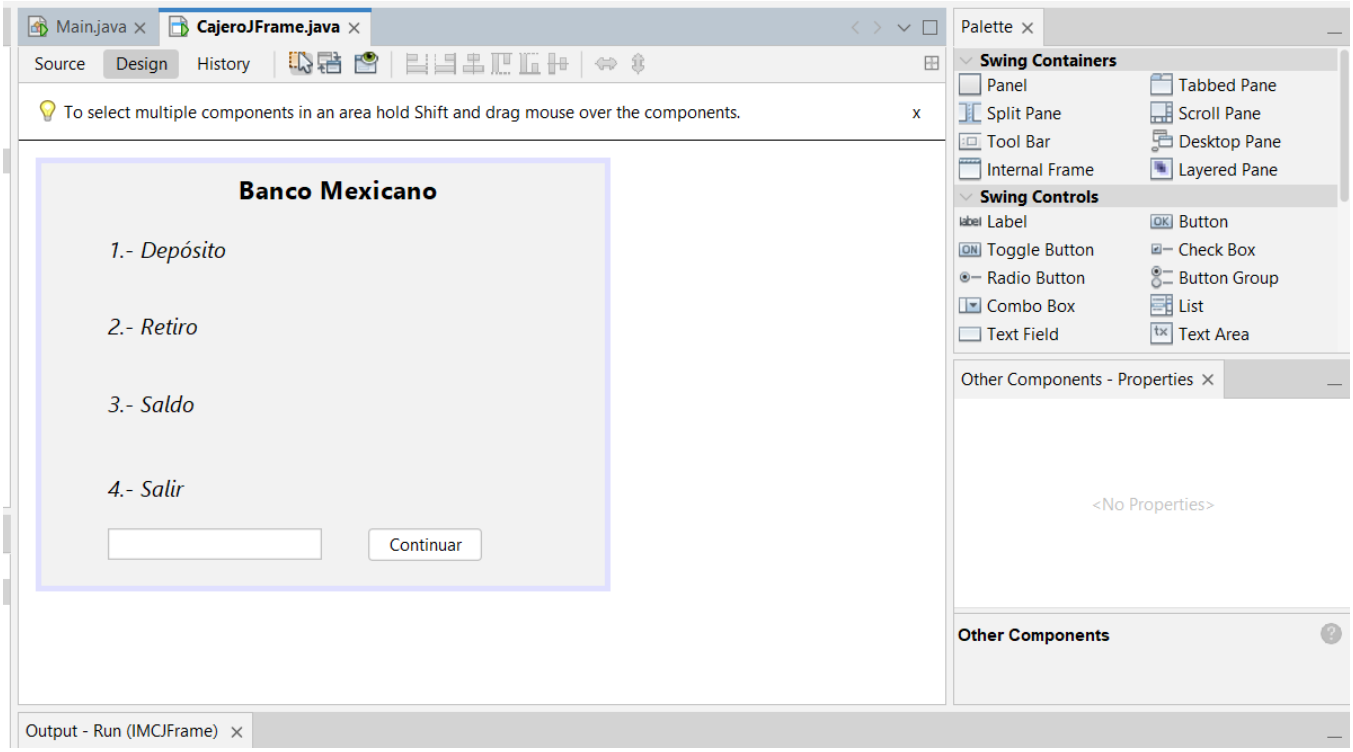
Al final, dominar estas reglas visuales es solo cuestión de práctica, pues la lógica subyacente que define a los objetos es universal.

Aunque la forma de escribirlo cambie, el concepto de Clase sigue siendo el mismo "molde" que en la tabla anterior.

Desarrollo

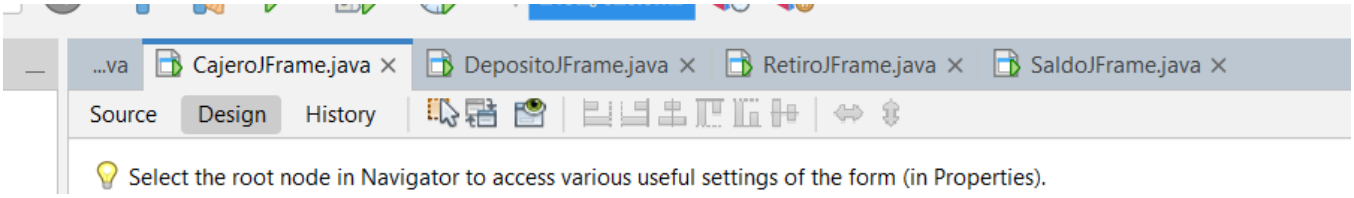


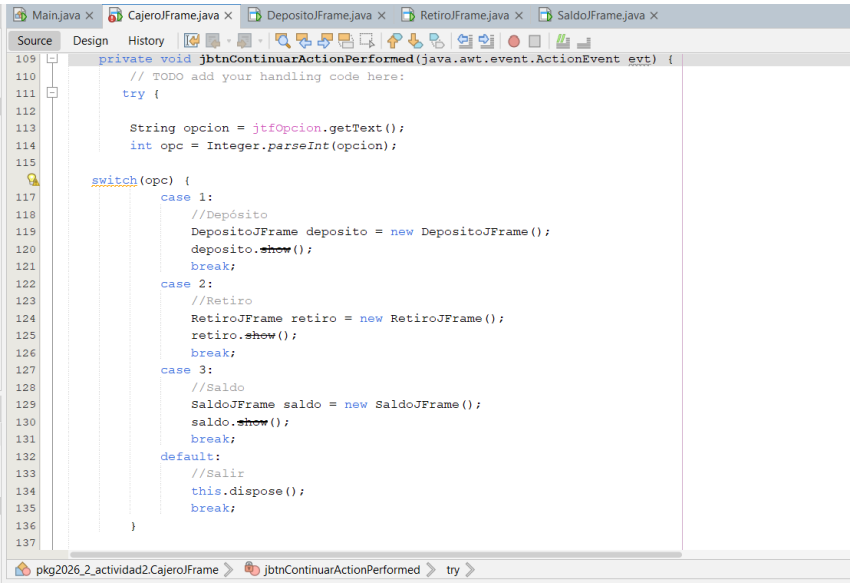
Creamos el entorno para nuestra actividad.



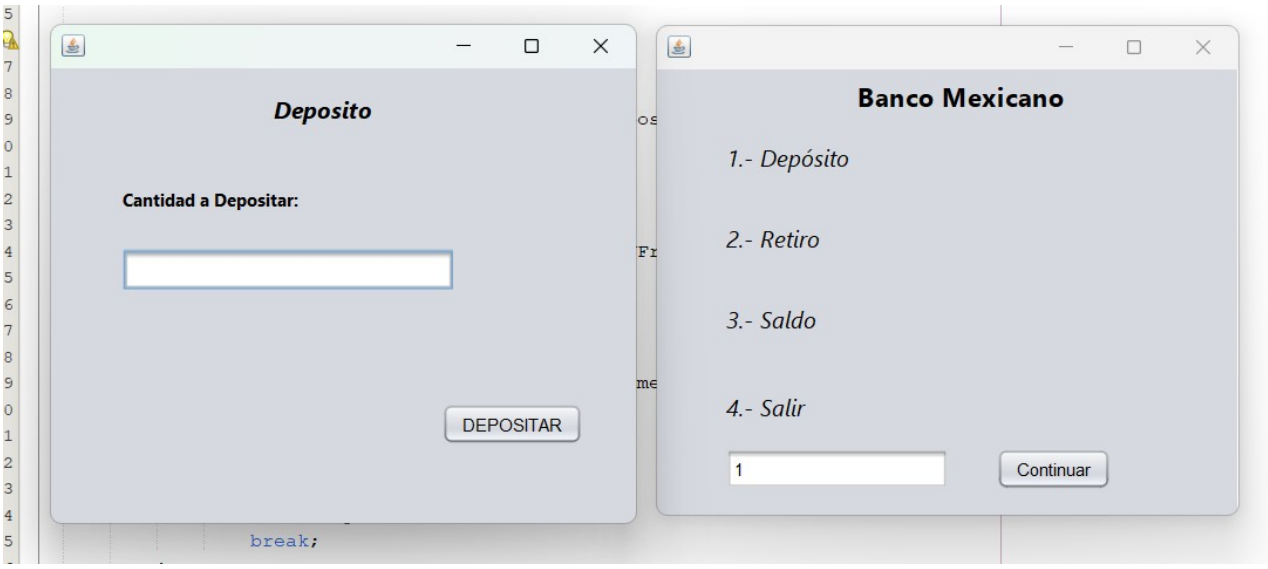
Creamos la interfaz principal donde estaremos trabajando.

Creamos nuestras formas:

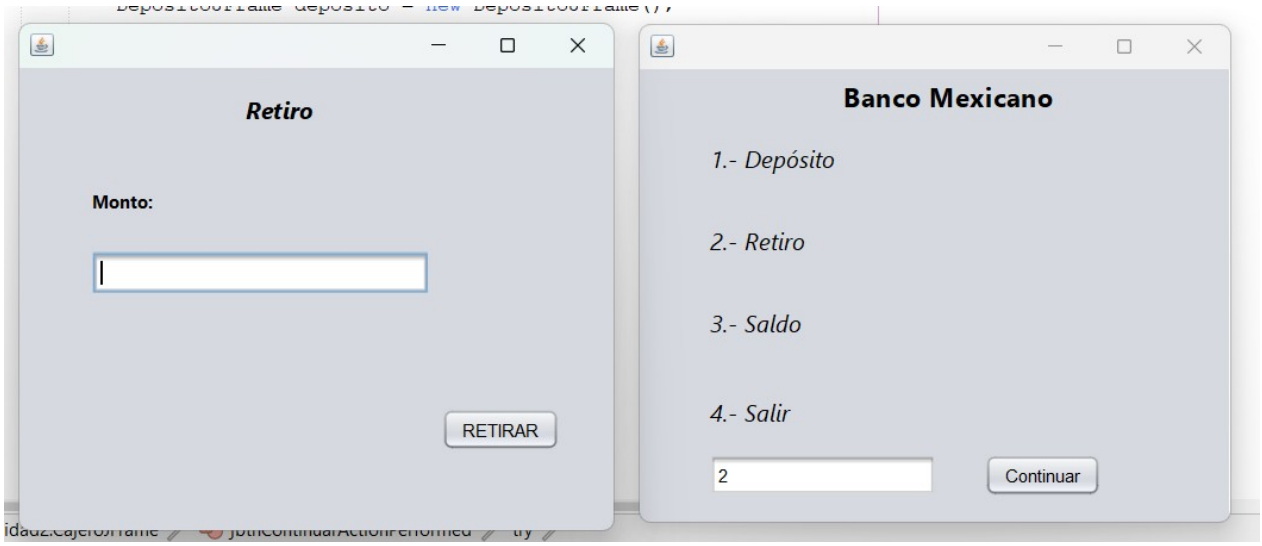




Metemos el código.



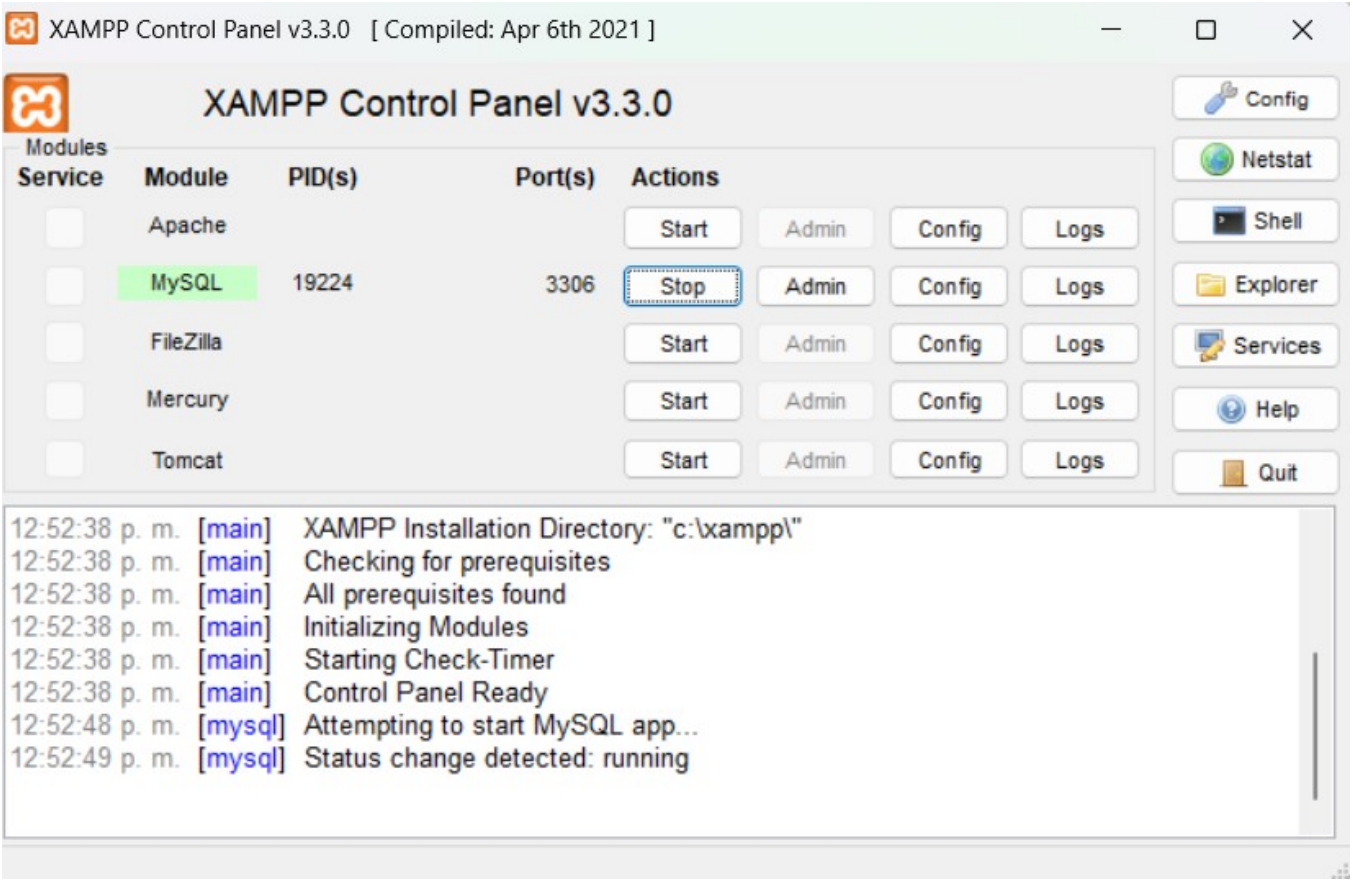
Funciona el Deposito.



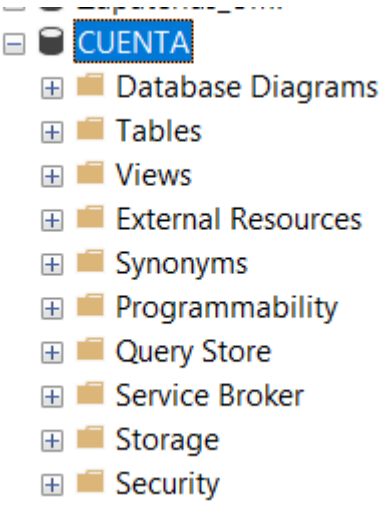
Funciona el Retiro.

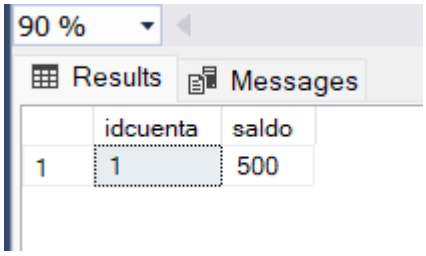
Funciona el Saldo.

Todas funcionan y se configuro la opción de “DISPOSE” en cada una para que no se cerrara todo el programa.



Nos conectamos a MySQL.





Creamos la base de datos.

```java
public class Conexion {

    public Connection getConnection() {
        Connection con = null;

        // Configuración para SQL Server según tu instancia local
        String baseDatos = "CUENTA";
        String url = "jdbc:sqlserver://localhost;instanceName=SQLEXPRESS;databaseName=" + baseDatos + ";encrypt=true;trustServerC

        try {
            // Cargar el driver de SQL Server
            Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");

            // Intentar establecer la conexión
            con = DriverManager.getConnection(url);

            if (con != null) {
                System.out.println("¡Conexión exitosa a SQL Server!");
            }

        } catch (Exception e) {
            // Mostrar error si la conexión falla
            JOptionPane.showMessageDialog(null, "Error de conexión: " + e.getMessage());
        }

        return null;
    }
```

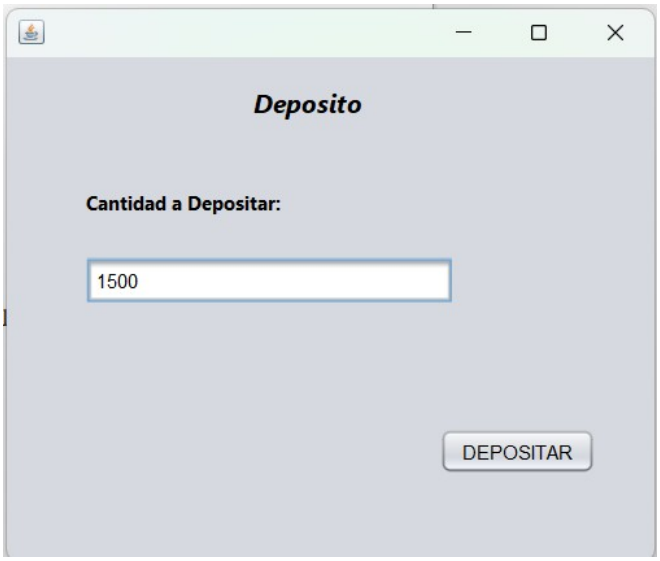
pkg2026_2_actividad2.Conexion > getConnection >

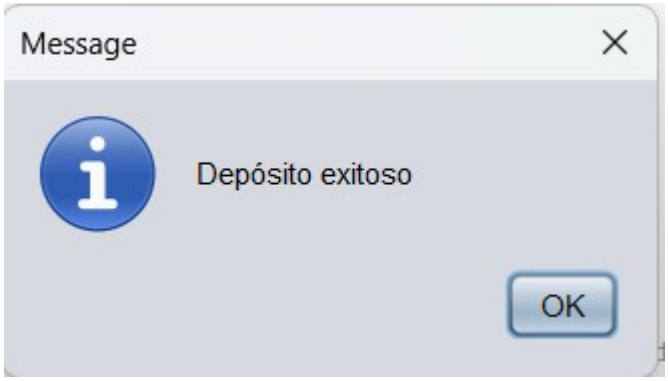
Hacemos la clase conexión adaptada a nuestro programa.

```java
    private void jbtnDepositarActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        Connection con;
        Conexion conn = new Conexion();

try {
    con = conn.getConnection();
    PreparedStatement ps = con.prepareStatement("UPDATE cuenta SET saldo = saldo + ? WHERE idcuenta = 1");

    // Convertimos el texto del campo a número para evitar errores de tipo de dato
    ps.setFloat(1, Float.parseFloat(jtfCantidad.getText()));

    int res = ps.executeUpdate();

    if (res > 0) {
        JOptionPane.showMessageDialog(null, "Depósito exitoso");
    } else {
        JOptionPane.showMessageDialog(null, "Error al depositar");
    }

    jtfCantidad.setText("");
    con.close();

} catch (Exception e) {


}
    }
```

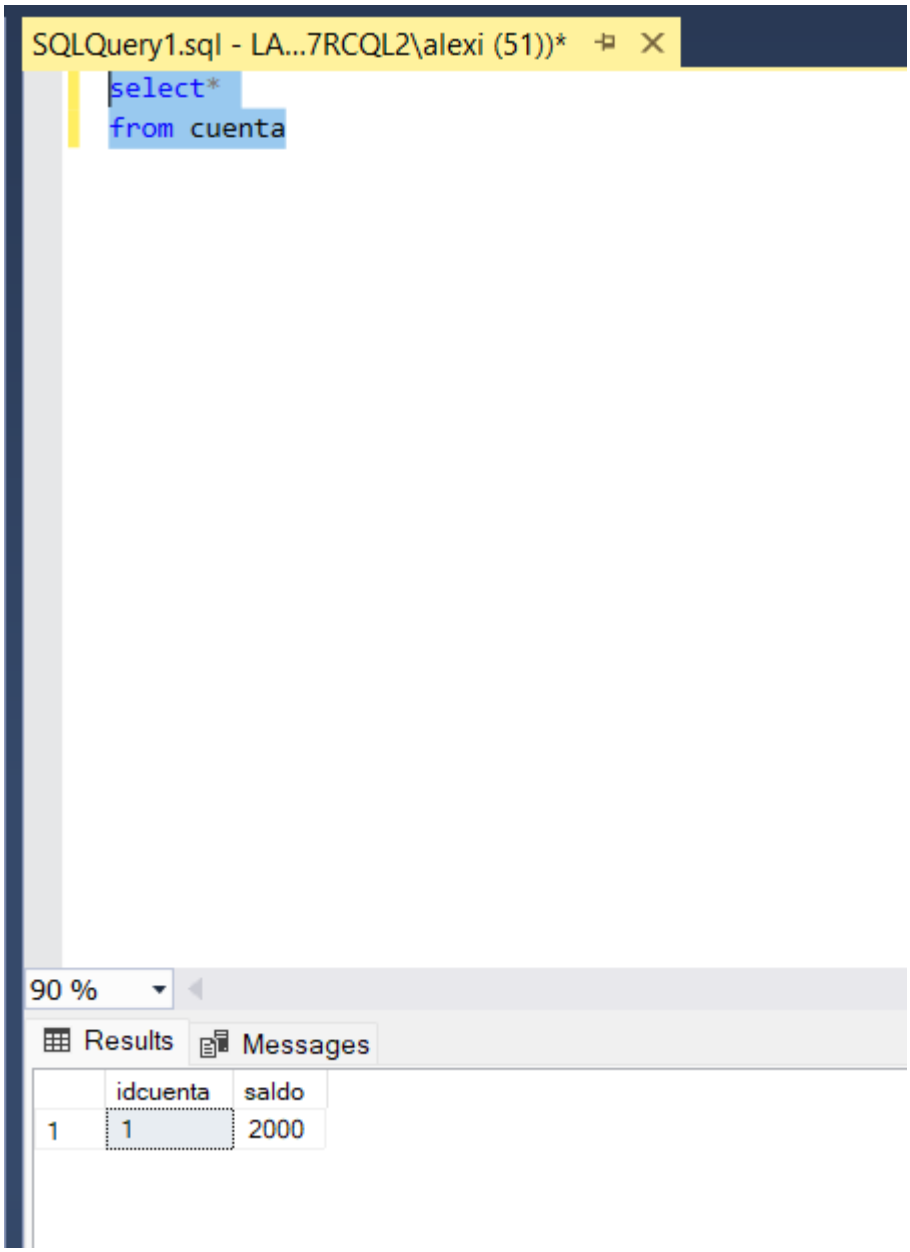
pkg2026_2_actividad2.DepositoJFrame > jbtnDepositarActionPerformed > try > catch Exception e >

Configuramos el deposito.





Fue muy complicado pero el final logre que se conectara a mis tablas de SQL.



Conclusión

Esta actividad permitió convertir una aplicación sencilla en un sistema mucho más completo y funcional.

Ya no se trata solo de una interfaz visual, sino de un programa capaz de guardar información real en una base de datos, lo que lo hace más cercano a una aplicación profesional.

A lo largo del proceso, se logró entender cómo conectar diferentes partes de un sistema: la interfaz en Java, la base de datos en SQL Server y los elementos necesarios para que ambos se comuniquen correctamente, como el driver JDBC y la configuración de red.

Cada paso fue importante para asegurar que los datos, como el saldo, se almacenaran de forma segura y permanente.

Además, esta actividad ayudó a desarrollar habilidades clave, como la resolución de problemas técnicos, la configuración de herramientas y la lógica para ejecutar acciones reales (como un depósito).

Referencias

Microsoft. (2022). ***Microsoft SQL Server 2022*** **(Versión 16.0) [Software de base de datos].** **https://www.microsoft.com/es-mx/sql-server/sql-server-downloads**

Apache Software Foundation. (2024). ***Apache NetBeans*** **(Versión 21) [Entorno de desarrollo integrado].** **https://netbeans.apache.org/** **}**

Microsoft. (2024). ***Microsoft JDBC Driver for SQL Server*** **(Versión 12.x) [Controlador de base de datos].** **https://learn.microsoft.com/es-es/sql/connect/jdbc/download-microsoft-jdbc-driver-for-sql-server**

Código GitHub:
https://github.com/AlexisRodriguez1988/Programaci-n-IV/tree/main